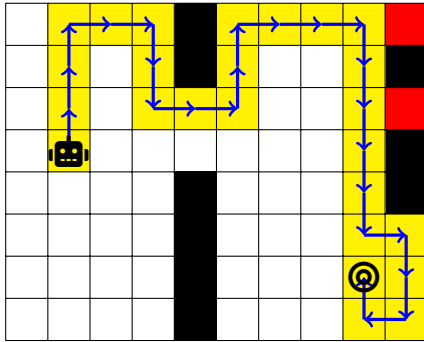


INTELLIGENT SYSTEMS

1ST MIDTERM EXAM, 14 NOVEMBER 2025

Task 1

Figure 1 shows a robot in a grid-like maze. The state of the robot is defined as pair (x, y) . The robot can move in four directions: up $(y - 1)$, down $(y + 1)$, left $(x - 1)$, right $(x + 1)$. Black blocks are obstacles through which the robot cannot pass. The icon of the robot represents the starting position, and the double circle the goal position.



```
actions ← [(0, -1), (1, 0), (0, 1), (-1, 0)]
```

```
Function transitions(current_state):  
    (x, y) ← current_state
```

```
    successors ← []  
    for each (dx, dy) in actions:  
        next_state ← (x + dx, y + dy)  
        if next_state not in obstacles:  
            if next_state not in visited_states:  
                successors ← successors + [next_state]  
  
    return successors
```

Figure 1: A robot in a maze and its transition model.

The transition model is described by the pseudocode next to the figure. Global lists **obstacles** and **visited_states** contain locations of obstacles and the states already visited by the search algorithm.

- (a) Draw the path that the robot makes if it uses depth-first search to navigate from the start to the goal position. Does the robot find an optimal path?

Marked with yellow color in the figure. The robot did not find an optimal path.

- (b) If depth-first search backtracks, mark the positions of backtracking. If it does not, explain why not.

The algorithm backtracked on two places, which are marked with red color in the figure.

- (c) Will depth-first search find a solution if we change the order of actions in the **actions** list? If yes, for which orders? Explain.

DFS will always find a solution, if it exists, no matter which order of actions is considered.

- (d) What is the length of the path found by breadth-first search? Defend your answer.

BFS always finds the shortest path, therefore the length of the found path will be 10.

Task 2

- (a) Suppose that it takes a computer 10 ms to process (visit) one node within a search tree. After running iterative deepening on a tree-like space with branching factor 3, the computer found the first solution after 1.7 seconds. At which depth d was this solution found? Include all your computations and comment on your reasoning.

Let N_i be the total number of nodes in the tree of depth i . This can be computed as $N_0 = 1$, and $N_i = N_{i-1} + 3^i$, so we have $N_i : 1, 4, 13, 40, 121, \dots$

The number of nodes ID_i that ID has to process in the worst case, if the solution is at depth i , can be computed as $ID_0 = 1$, and $ID_i = ID_{i-1} + N_i$. We have: $ID_i : 1, 5, 18, 58, 179, \dots$

The worst-case time T_i needed for ID to find the solution at depth i is $T_i = ID_i \cdot 0.01$ s. We have $T_i : 0.01$ s, 0.05 s, 0.18 s, 0.58 s, 1.79 s.

The solution was found at depth $i = 4$.

- (b) Suppose we now run breadth-first search on that same space. How much time will it now take to find that same solution? Give the exact answer, not just an approximation.

Iterative deepening visited 170 nodes before finding a solution. This means it stopped 9 nodes before the whole 4-th level would be searched. Understanding the manner in which a tree is traversed with DFS/ID, we can deduce that 7 nodes on the 4-th level have not been visited in the final iteration. The explanation is given in Figure 2, where yellow nodes are those visited by ID in the last iteration, the red node is the found solution, and the white nodes are not visited. BFS therefore visits $121 - 7 = 114$ nodes before finding a solution, which takes 1.14 seconds.

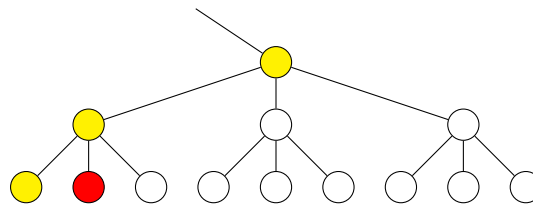


Figure 2: Nodes visited in the last ID iteration. The image shows only the final stages of search, the whole space is much bigger.

Task 3

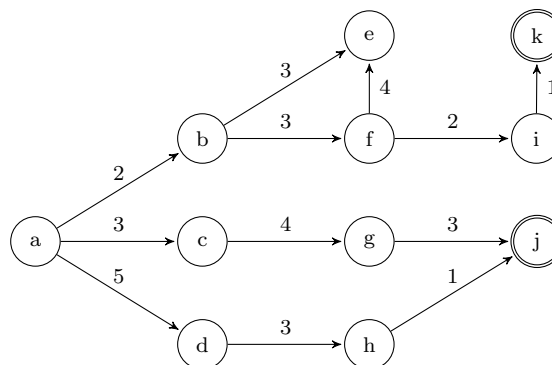
A state transition model is given by the below Prolog clauses.

```
link(a,b,2). link(a,c,3). link(a,d,5). link(b,e,3). link(b,f,3). link(c,g,4).
link(d,h,3). link(f,e,4). link(f,i,2). link(g,j,3). link(h,j,1). link(i,k,1).
start(a). goal(j). goal(k).
```

The arguments of the `link/3` predicate are (*source*, *destination*, *cost*). Additionally, a heuristic function is given as:

```
h(a, 8). h(b, 4). h(c, 3). h(d, 3). h(e, 8). h(f, 1).
h(g, 2). h(h, 1). h(i, 3). h(j, 0). h(k, 0).
```

- (a) Draw the state space.



- (b) Which solution does the A* algorithm find? Assume that the priority queue sorts the elements with the same value f alphabetically. Write down the order in which the algorithm visits the states.

Solution: $a \rightarrow d \rightarrow h \rightarrow j$, with the cost of 9.

The order of visiting the states is: a, b, c, f, d, g, h, j.

- (c) Is the found solution optimal? If yes, can you guarantee that for any start/goal setup? If not, suggest a minimal correction of the above Prolog database so that A* always finds the optimal solution.

The found solution is not optimal, because A* with the given heuristic function is not admissible, since $h(i) > h^*(i)$.

We may replace the Prolog predicate `h(i, 3)` with `h(i, 1)` to make the heuristic admissible. Then, the found solution is: $a \rightarrow b \rightarrow f \rightarrow i \rightarrow k$, with the cost of 8.

- (d) Write a Prolog predicate `states(L)`, where L is the set of all the states (a list without duplicates).

```
state(S) :- link(S, _, _); link(_, S, _).
states(L) :- setof(S, state(S), L).
```

Task 4

Figure 3 shows a decision tree of a deterministic, fully observable, turn based two-player game, where node *a* represents the current state, and it is the AI agent's turn to decide on the next move. The square nodes represent the terminal states, in which the game is finished. A positive value in a terminal state means the AI agent won the game, a negative value means it lost the game, zero signifies a draw.

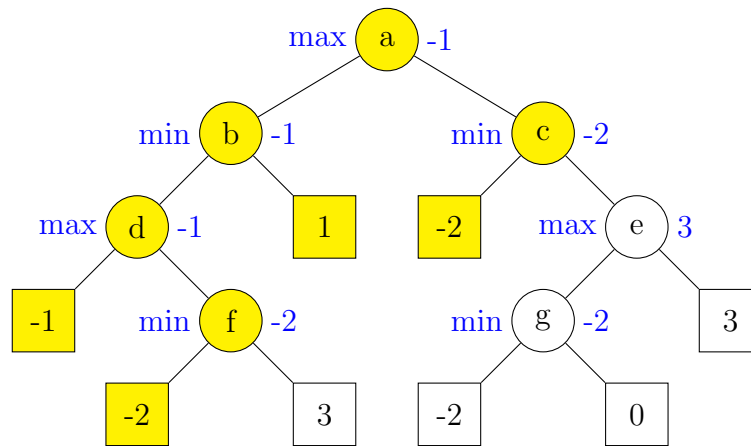


Figure 3: An adversary state space. Empty nodes get pruned by alpha-beta pruning.

- (a) Can the AI agent win the game if it uses the minimax algorithm to decide on the next move? Assume that both players play optimally. Explain.

No, the AI cannot win the game. If it chooses branch *b*, the maximum score it can achieve is -1, and if it chooses branch *c*, the maximum score is -2. There is no move that allows the AI to achieve a positive outcome.

- (b) Suppose the minimax algorithm takes the same amount of time to process each node. How much faster (in percentage) does the agent decide on the move if alpha-beta pruning is used? Explain.

The empty nodes in Figure 3 are pruned by alpha-beta pruning. This means that the algorithm prunes $\frac{6}{15} = \frac{4}{10} = 40\%$ of the states. Since each state takes the same amount of time to be processed, that is also the increase in the speed to decide the next move.

- (c) Suppose the tree in Figure 3 is defined by a Prolog predicate `link(Parent, Child)`, where `Parent` and `Child` are nodes. A node is either a letter *a*, ..., *g*, or a terminal value -2, -1, 0, ... Write a Prolog predicate `contains_draw(Root)`, where `Root` is any given node, and the predicate is true if that subtree contains the terminal value 0. For example, `contains_draw(a)` is true, while `contains_draw(b)` is false.

```
contains_draw(Node) :- link(Node, 0).
contains_draw(Node) :- link(Node, Child), contains_draw(Child).
```